YOUR FITNESS COMPANION

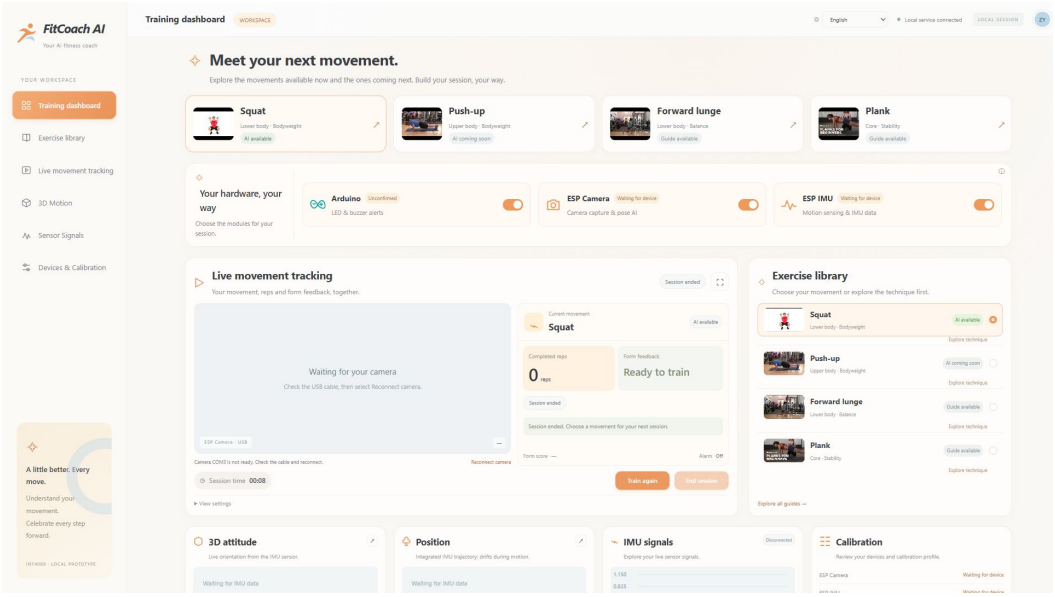
Product Assembly Guide

and User Manual

Sense your movement. Support every repetition.

01 / VISION 02 / MOTION 03 / FEEDBACK

Your training workspace FITCOACH AI



Actual interface capture / disconnected-device state

Before your first session

Build the camera, motion and alarm modules step by step. Live BMI160 acquisition and physical light/sound acceptance in the latest loop remain pending. Illustrations describe connections, not PCB layouts.

Zhang Yancheng

Version 1.1 / 11 November 2026

Integration baseline: `9152fe896a05` / Assembly, setup and everyday operation

Contents

1 Product overview and readiness 2
1.1 Intended use . . . 2

2 Parts and preparation 3
2.1 Before starting . . . 3
2.2 Visual parts reference: controllers and sensor . . . 4
2.3 Visual parts reference: cables and small components . . . 5

3 System connections at a glance 7

4 Assembly: motion sensor 8
4.1 Connect the BMI160 with power removed . . . 8
4.2 Mount and verify . . . 8

5 Assembly: alarm and camera 9
5.1 Connect the one-way alarm interface . . . 9
5.2 Install the camera . . . 9

6 Software installation and firmware 10
6.1 Prepare a new PC . . . 10
6.2 Upload only when needed . . . 10
6.3 Launch the unified application . . . 10

7 Daily operation 11
7.1 Start a session . . . 11
7.2 Camera direction and teaching previews . . . 11

8 Understanding feedback and motion views 12
8.1 Repetitions, scores and alarms . . . 12
8.2 Live IMU, replay and calibration . . . 12
8.3 End the session . . . 12

9 Commissioning and troubleshooting 13
9.1 Commissioning checklist . . . 13
9.2 Common problems . . . 13

10 Maintenance and technical reference 14
10.1 Care and records . . . 14
10.2 Alarm interface for maintainers . . . 14
10.3 Evidence and source map . . . 14

1 Product overview and readiness

FitCoach AI helps demonstrate camera-based squat assessment. An ESP32-CAM sends images to a PC, where YOLOv8n-pose estimates body keypoints and geometric rules evaluate movement. A separate ESP32-S3 relays alarm states to an Arduino. A BMI160 provides a motion-sensing development path and dashboard visualizations.

What the current product supports

The unified application enables **Squat AI**. Push-up and Forward lunge have teaching previews marked AI coming soon. Legacy push-up rules in the source do not make push-up AI available in the current unified launcher.

Subsystem	Readiness at this revision
Vision	Real USB frames and PC inference validated; real exercise accuracy and thresholds still need evaluation.
Wireless bridge	ON/OFF, acknowledgements and command expiry validated. An ACK proves S3 receipt only.
IMU	Current hardware does not answer at either I^2C address. Historical six-axis success does not establish present health.
Physical alarm	Protocol reception has evidence; latest-loop Arduino reception, visible LED and audible buzzer still need on-site acceptance.
Cloud	Future direction; no cloud deployment is required.

1.1 Intended use

Use as a supervised educational prototype in a clear indoor exercise area. Feedback is a rule-based estimate, not a medical assessment or a guarantee of safe form. Stop movement if pain, dizziness or equipment problems occur. Do not rely on the alarm as a protective interlock.

2 Parts and preparation

Part	Qty	Purpose / selection note
ESP32-CAM + OV2640	1	Camera node, supported USB programming/serial connection.
ESP32-S3 N16R8	1	16 MB flash / 8 MB PSRAM target; radio relay and IMU host.
BMI160 breakout	1	Existing labelled module; verify pin labels before applying the wiring table.
Arduino	1	Local alarm controller.
LED + resistor	1 each	Series 220 Ω resistor required.
Passive buzzer	1	Firmware uses 1600 Hz tone; use a GPIO-compatible low-current device.
USB data cables	3	One each for CAM, S3 and Arduino. Must support data transfer, not charging only; match each actual socket.
Jumper wires / breadboard	As needed	Secure short connections; verify split power rails.
PC and camera mount	1 each	Python environment, USB ports and stable framing.

2.1 Before starting

1. Identify every board by its printed label. Record CAM, S3 and Arduino serial ports separately; COM3 and COM4 are examples from the development PC.
2. Inspect cables, loose conductors and exposed pins. Disconnect **all** USB supplies before adding or moving wires.
3. Keep the repository layout intact: integrated S3 firmware includes the BMI160 source through a relative path.
4. Use the active sketches under `firmware/usb-ai-bridge/` and `firmware/alarm-mega/`. Historical test sketches replace the active firmware when uploaded.

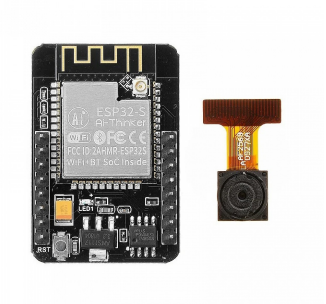
Power and logic levels

Power each controller through its own USB connection. Do not join controller supply pins. Join S3 and Arduino grounds for UART. Never connect Arduino D18 TX to the S3: a reverse 5 V signal needs suitable level conversion. The existing 3.3 V S3-to-Arduino link passed reception tests but is not qualified across worst-case logic tolerances.

2.2 Visual parts reference: controllers and sensor

These are identification references, not photographs of the assembled prototype. Connector type, PCB layout and pin order may differ by board revision; use the wiring tables and the markings on your own hardware.

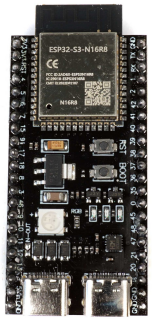
01 / ESP32-CAM + OV2640



Camera node. The camera is shown detached; this board also needs a compatible USB programming/serial interface.

Source: FlyRobo product photograph

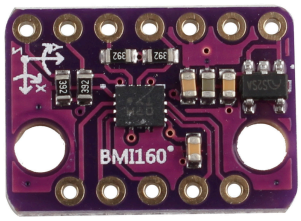
02 / ESP32-S3 N16R8



Motion host and alarm relay. N16R8 identifies memory configuration; it does not uniquely identify a PCB or USB connector.

Source: Elektroweb product photograph

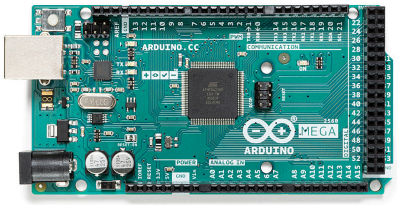
03 / BMI160 breakout



Six-axis IMU. Purple breakout appearance is representative; verify 3v3, VIN, CS and SA0 labels on the actual unit.

Source: HALJIA product photograph

04 / Arduino



Alarm controller. The reference board has a USB Type-B socket. Compatible boards can use another connector.

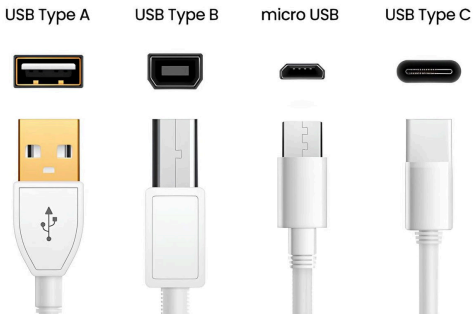
Source: Arduino official product photograph

Photographs belong to their respective sources and are reproduced here as component identification references. They do not establish electrical compatibility with the prototype.

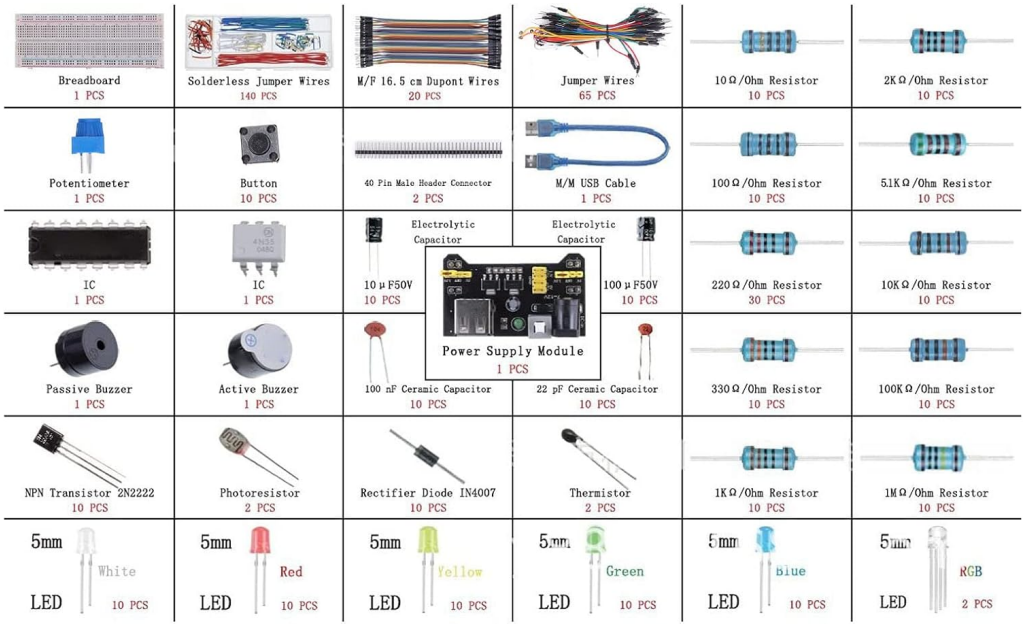
2.3 Visual parts reference: cables and small components

Three USB data cables are required

Use one data-capable USB cable for each controller: CAM (through its USB interface), S3 and Arduino. Match the PC end and device end separately. The S3 reference uses USB-C and the Arduino reference uses Type-B; confirm the actual CAM adapter socket. Connector shape alone cannot distinguish a charge-only cable from a data cable. Confirm that each board appears as a serial device.



USB connector identification. Type-A, Type-B, Micro-USB and Type-C appearance examples. Source: UGREEN USB connector guide.



Small-component identification. Reference chart showing breadboard, jumper wires, LEDs, resistors and passive/active buzzers. Use only the parts listed in this manual: one LED, one 220 Ω resistor, one compatible low-current passive buzzer, breadboard and jumper wires. Other chart items and quantities are not required. Source: Psytfei kit reference chart.

Check values, not appearance alone

Measure or read the resistor value; colour and body shape are insufficient. Active and passive buzzers can look alike, so check their type and current rating. Select jumper ends to match the

actual headers. The PC and stable camera support are additional equipment, not illustrated electronic components.

3 System connections at a glance

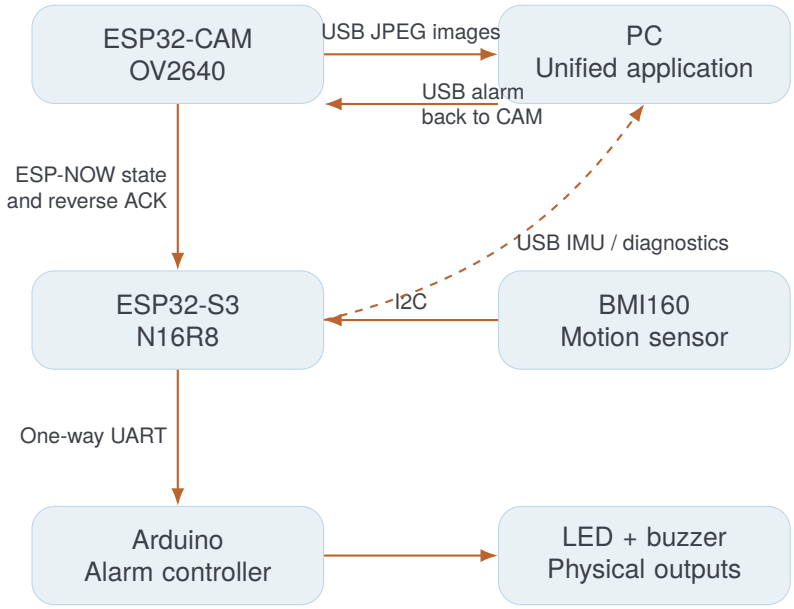


Figure 1. Functional data paths. Each controller also has independent USB power. Dashed USB telemetry is separate from the alarm radio link; images never travel over ESP-NOW.

Link	Fixed configuration
CAM to PC	921600 baud, 8N1; 640 × 480 JPEG, quality 18, two PSRAM buffers, LATEST mode.
S3 USB	115200 baud, 8N1; IMU CSV and diagnostics.
CAM to S3	ESP-NOW channel 1; 20-byte state/ACK packets, 200 ms interval, fixed peers.
S3 to Arduino	9600 baud, 8N1; GPIO17 TX to D19 RX1.

Replacement boards

Verify the peer MAC addresses in `firmware/usb-ai-bridge/AlarmRadio.h` when replacing either ESP32. The CAM target currently expects S3 peer `28:84:85:43:72:B0`; S3 expects CAM peer `E8:6B:EA:CA:EE:88`. These are current firmware settings, not universal board addresses.

4 Assembly: motion sensor

4.1 Connect the BMI160 with power removed

This table applies to the existing breakout with separate `3v3`, `VIN`, `CS` and `SA0` labels. Another breakout may have a different regulator or pinout. Follow labels rather than wire colour, physical pin count or a mirrored photograph.

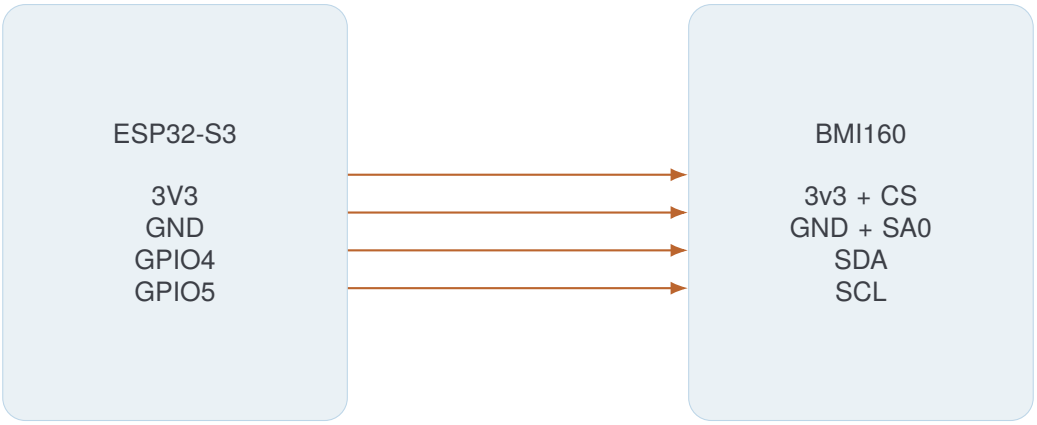


Figure 2. Signal-group illustration; the table below is the authoritative connection list. Box text is not a physical pin arrangement.

S3	BMI160	Function
3V3	3v3	Supply the module's 3.3 V rail; leave VIN open.
GND	GND	Common reference.
GPIO4	SDA	I2C data.
GPIO5	SCL	I2C clock.
3V3	CS	Hold high for I2C operation.
GND	SA0	Select address 0x68.

Leave VIN, OCS, INT1, INT2, SCX and SDX unconnected in this arrangement. CS is not OCS; SDA/SCL are not SDX/SCX. Check breadboard rail continuity before power-up.

4.2 Mount and verify

Secure the module without stressing its leads. Record the sensor's axis orientation; body placement and a validated sensor-to-exercise fusion method have not been established. Keep loose bench wiring off the exercising person.

After power-up, an identity check should find `CHIP_ID=0xD1` at the selected address. The active firmware probes 0x68 and 0x69. **Both addresses currently fail on the integrated hardware.** Power down and inspect supply, continuity, CS/SA0 and I2C pull-ups before claiming live motion sensing. A replay trace is not a substitute for this check.

5 Assembly: alarm and camera

5.1 Connect the one-way alarm interface



Figure 3. UART carries alarm commands only. No return acknowledgement wire exists from Arduino to S3.

1. With both boards unpowered, connect S3 GPIO17 to Arduino D19 and S3 GND to Arduino GND. Leave Arduino D18 and S3 GPIO18 open.
2. Connect Arduino D8 through a 220 Ω series resistor to the LED anode; connect LED cathode to Arduino GND. Do not omit the resistor.
3. Connect a compatible low-current passive buzzer positive terminal to D9 and negative terminal to GND. Check its rating first; a higher-current buzzer needs a suitable driver circuit.
4. Inspect for shorts. Connect each board's USB supply independently. Both LED and buzzer should start OFF.

5.2 Install the camera

Secure the ESP32-CAM and its USB interface to a stable support. Keep the lens and ribbon connection unobstructed; disconnect power before reseating the camera. The built-in OV2640 wiring is defined by the active CAM sketch, not by the S3 IMU table.

Start at roughly 2–3 m from the user, then adjust so the entire body, including ankles, stays visible throughout the squat. Use even front lighting and an unobstructed exercise area. A side or approximately 45-degree view reveals torso lean better than a pure front view; left/right symmetry is harder to observe from a pure side view.

Acceptance boundary

An ESP-NOW ACK confirms S3 receipt, not LED illumination or buzzer sound. Check both outputs directly during commissioning. Startup-OFF and communication expiry are implemented behaviors, not safety-rated protection.

6 Software installation and firmware

6.1 Prepare a new PC

Install Python and use a dedicated environment. From the repository root:

```
python -m venv app/.venv
app/.venv/Scripts/python.exe -m pip install -r app/requirements.txt
app/.venv/Scripts/python.exe -m serial.tools.list_ports
```

The requirements specify dependency ranges, not an exact reproducible lockfile. Prepare the official `yolov8n-pose.pt` weights before an offline demonstration. First-time package and weight retrieval needs internet access. A configured development PC can reuse its existing environment.

6.2 Upload only when needed

If the active firmware is already installed, do not reflash for normal use. For a fresh build, install Arduino CLI, the Arduino AVR board package and Espressif ESP32 board package. Close the application, serial monitors and test tools first. Confirm board and port identities.

```
arduino-cli compile --fqbn esp32:esp32:esp32cam firmware/usb-ai-bridge/Cam_USB_Bridge
arduino-cli compile --fqbn esp32:esp32:esp32s3:FlashSize=16M,PSRAM=opi,CDCOnBoot=
    default,USBMode=hwcdc,PartitionScheme=app3M_fat9M_16MB,UploadSpeed=115200 firmware/
    usb-ai-bridge/S3_IMU_Alarm_Bridge
arduino-cli compile --fqbn arduino:avr:mega firmware/alarm-mega/Mega_Alarm_RX
```

Append `--upload--port<actual-port>` to the matching command, replacing the placeholder. Uploading replaces the board's existing program. Keep the shared header and included BMI160 source at their original relative locations.

6.3 Launch the unified application

From the repository root; substitute your actual CAM and S3 ports:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File app/run_pose.ps1 -CamPort COM3 -
    ImuPort COM4 -Action squat
```

Open `http://127.0.0.1:8010`. To skip automatic IMU connection, use `-ImuPort''`. The active launcher accepts squat only. Older README examples for pushup or standalone entry points do not describe the current unified experience.

7 Daily operation

7.1 Start a session

1. Inspect the assembly and clear the exercise area. Connect the CAM, S3 and Arduino USB cables. Identify ports again after reconnecting.
2. Launch the application and open the local page. Choose English in the header language selector; the browser remembers the preference.
3. Confirm a fresh camera image and check connection indicators. A radio ACK alone does not establish IMU or actuator health.
4. Stand upright briefly with shoulders, hips and naturally lowered hands visible. Automatic camera direction needs three consistent, sufficiently distinct scans before locking.
5. Select Squat AI from the movement library. Keep the whole body visible and perform a slow trial repetition before a demonstration.

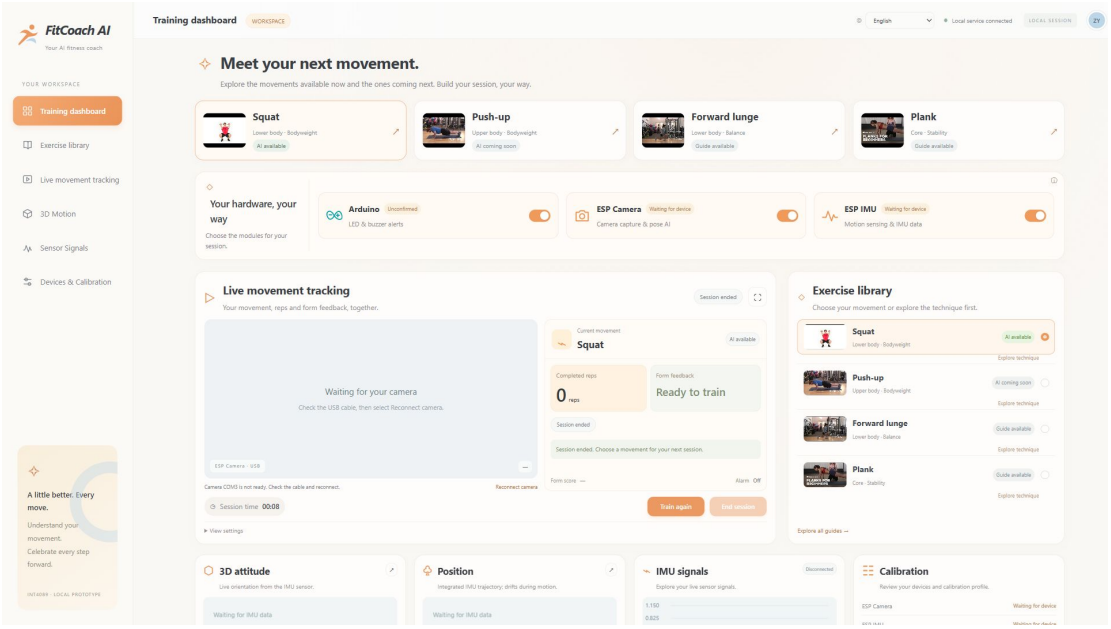


Figure 4. Repository English dashboard capture. Devices are disconnected in this screenshot; it illustrates the interface, not a successful live session. The four views provide overview, 3D attitude/position, signal charts and connections/replay/calibration.

7.2 Camera direction and teaching previews

The default view uses automatic quarter-turn correction and selfie mirroring. After moving the camera, use **Recalibrate direction**. If automatic orientation is ambiguous, assessment and alarms pause until calibration succeeds; fixed 0/90/180/270-degree overrides are available. Disable mirroring for a conventional or rear-facing view.

Teaching previews explain three movements and may load external videos when clicked. They do not change the AI exercise or alarm state. Push-up and lunge previews are not active AI modes.

8 Understanding feedback and motion views

8.1 Repetitions, scores and alarms

A repetition is counted on return from descent to standing. The rules track the movement rather than judging a single still frame. A completed repetition with a rule violation produces a two-second alarm event. Missing required keypoints prevents assessment and clears the alarm; no alarm does not prove correct form.

Squat metric	Current rule	Interpretation
Minimum knee angle	At most 105 degrees	Larger minimum angle is labelled insufficient depth.
Maximum torso tilt	At most 45 degrees	Larger tilt is labelled excessive forward lean.
Left/right knee difference	At most 20 degrees	Larger difference is labelled asymmetry.

These are implementation thresholds in `app/form_rules.py`, not validated individual exercise prescriptions. The score is a rule score, not model confidence or an injury-risk percentage. Perspective, occlusion and keypoint jitter affect results; monocular images do not provide a complete 3D biomechanical assessment.

8.2 Live IMU, replay and calibration

1. In Connections, select the S3 port, not the CAM port. Default connection avoids resetting S3; an explicit reset briefly interrupts its radio relay.
2. Check the source label and valid sample status. “Live” requires real parsed samples; a connection alone does not prove sensor health.
3. For software demonstrations, load a historical or uploaded CSV and use pause/resume. Replay affects IMU visualization only; live vision continues independently and may still trigger its normal alarm.
4. Import or edit a profile, or calibrate from stationary CSV, only with a trace whose stationary conditions are known. Export JSON to retain changes; session edits do not overwrite repository profiles.

The 3D view displays attitude, gravity, velocity and an integrated position trail. Position drifts during motion because acceleration is integrated twice. Yaw has no magnetometer reference. Position reset changes the origin; it does not improve physical accuracy. Replay never proves current hardware health and does not generate alarms.

8.3 End the session

Stop exercising, stop the server with Ctrl+C, and verify outputs are OFF. On command loss, CAM/S3 leases expire after about 1.2 seconds and Arduino turns outputs off after 1.5 seconds without an accepted frame. These are separate timers, not a guaranteed total shutdown latency. Disconnect all supplies before storage or rewiring.

9 Commissioning and troubleshooting

9.1 Commissioning checklist

Record date, code revision, board identities, ports and observations. Keep compilation, upload, communication and physical acceptance as separate results.

Check	Pass evidence to record
Power-off inspection	Correct six IMU wires; one-way UART; no joined supply pins; resistor and buzzer rating checked.
Camera	Fresh decoded frames with whole-body framing and stable orientation.
Vision	Trial repetition counted; feedback matches observed movement. Real accuracy still needs a labelled evaluation.
Radio	ON/OFF and S3 ACK observed; document command-expiry behavior.
Actuators	Arduino receives commands; LED visibly changes and buzzer audibly changes in the latest loop.
IMU	Expected identity and plausible live six-axis samples; current unit remains pending.
Loss cases	Observe actual USB/radio/power-loss behavior; do not infer it solely from software tests.

9.2 Common problems

Symptom	Action
Port busy / upload fails	Stop the app, serial monitor and standalone tools. Only one process can own a port.
No camera image	Check USB data cable and actual CAM port. Use camera reconnection in the page; re-identify the port after reconnecting.
No repetitions	Check full-body framing, lighting, Squat selection and orientation lock. Ankles outside the image prevent knee-angle assessment.
S3 ACK absent	Check both nodes are powered, correct active firmware, fixed peer MACs and channel 1. Images may still work over USB.
ACK present, no alarm	Check S3 GPIO17 to Arduino D19 and common ground. Check Arduino firmware, LED polarity, resistor and buzzer rating. ACK is not actuator feedback.
IMU has no samples	Power off; inspect 3.3 V rail, ground, CS/SA0, SDA/SCL, continuity and pull-ups. Both addresses currently fail on the unit.
Position drifts	Expected integration limitation; reset origin for viewing. Do not interpret the trail as measured absolute position.

10 Maintenance and technical reference

10.1 Care and records

Store the assembly dry and unpowered; protect the lens and avoid pulling sensor leads. Before each demonstration, inspect connections and repeat the basic camera/actuator checks. Keep credentials and identifiable camera recordings out of version control. The unified integration does not save raw camera frames by default.

10.2 Alarm interface for maintainers

Arduino accepts exactly three ASCII binary digits plus LF (CRLF accepted): source, state, source-XOR-state. PC USB is source 0 at 115200 baud; S3 UART is source 1 at 9600 baud.

Frame + LF	Input	Meaning
`000`	PC USB	OFF
`011`	PC USB	ON
`101`	S3 UART	OFF
`110`	S3 UART	ON

For a supervised direct Arduino test, stop competing tools, open its actual port in Serial Monitor at 115200 with newline ending, wait about two seconds after opening, then send `011` and `000`. A single accepted command lasts only 1.5 seconds; a 200 ms heartbeat sustains control. Direct PC commands take priority over S3 for 1.5 seconds. An OFF command is not a physical disconnection of power.

10.3 Evidence and source map

This manual was checked against integration snapshot `9152fe896a05`. It is not based on the older main-branch plan. Relevant repository files:

- `README.md`; `app/UNIFIED_APP.md`; `app/run_pose.ps1`: current architecture, interface and launch behavior.
- `firmware/usb-ai-bridge/`: active CAM/S3 sketches and `AlarmRadio.h`; `firmware/alarm-mega/Mega_Alarm_RX/`: actuator code.
- `firmware/esp32-camera/BMI160_WIRING.md`: existing breakout wiring and later no-response findings.
- `docs/protocols/`: USB, ESP-NOW and alarm contracts.
- `docs/validation/2026-11-10-camera-speed-after.json`: USB-only 268 frames in 30 seconds, 8.93 FPS, no CRC/decode errors. This is not guaranteed AI throughput.
- `docs/validation/2026-11-11-unified-app.json`: unified application validation scope. Real exercise accuracy, live IMU and physical latest-loop acceptance remain open.